MODELARE ȘI SIMULARE

Săptămâna 8. Automate celulare, particule, simulare de înaltă performanță

Suport de curs · Anul III, Calculatoare și Tehnologia Informației

Ș.l. dr. ing. Cătălin Iapă

Obiective

1. să definiți un automat celular prin rețea, stări, vecinătate și regulă locală și să explicați actualizarea sincronă;
2. să simulați automate elementare 1D și Game of Life, vectorizat cu NumPy, și să explicați emergența și clasele lui Wolfram;
3. să construiți și să interpretați modele CA pentru trafic (Nagel–Schreckenberg), incendii și percolare, creștere și difuzie;
4. să explicați ideea gazelor pe rețea și a metodei Lattice Boltzmann (coliziune + propagare, legi de conservare);
5. să integrați un sistem de particule cu metoda Verlet și să reduceți costul forțelor cu raza de tăiere, grila și Barnes–Hut;
6. să estimați câștigul din vectorizare și paralelism (legea lui Amdahl) și să recunoașteți ce nu se poate paraleliza.

Cuvinte cheie: *automat celular, rețea, vecinătate von Neumann, vecinătate Moore, regulă locală, actualizare sincronă, condiții la margine, numerotarea Wolfram, clasele lui Wolfram, Game of Life, planor, emergență, ireductibilitate computațională, furnica lui Langton, mediu excitabil, regula 184, modelul Nagel–Schreckenberg, diagrama fundamentală, ambuteiaj fantomă, percolare, prag de percolare, difuzie, gaz pe rețea, HPP, FHP, Lattice Boltzmann, D2Q9, BGK, potențial Lennard-Jones, rază de tăiere, metoda Verlet, metodă simplectică, metoda grilei, arbore k-d, Barnes–Hut, vectorizare, memorie partajată, memorie distribuită, descompunerea domeniului, legea lui Amdahl, GPU, exascale*

1. Automate celulare

1.1. Definiție: rețea, stări, vecinătate, regulă

Până acum am modelat sisteme fără spațiu: cozi, populații omogen amestecate, ecuații cu câteva variabile. Un **automat celular** (CA) este, în cuvintele profesorului Cernazean, „un univers fictiv în care totul este discret”: spațiul este o rețea de celule, timpul avansează în pași, iar fiecare celulă are un număr finit de stări. Formal, un automat celular este un cvadruplu ⟨A, S, N, Φ⟩: rețeaua A (linie, grilă pătrată, rețea hexagonală), mulțimea de stări S, vecinătatea N a unei celule și regula locală Φ, care dă starea nouă a celulei în funcție de stările vecinilor. Toate celulele se actualizează **simultan** (sincron), pe baza stărilor de la pasul anterior.

Cele mai folosite vecinătăți pe grila pătrată sunt **von Neumann** (cei 4 vecini de pe laturi) și **Moore** (cei 8 vecini, inclusiv diagonalele). Grila finită cere **condiții la margine**: periodice (grila se închide ca un tor – cele mai folosite în simulări, pentru că elimină efectele de margine), fixe (celule moarte sau cu valoare impusă) sau reflectante. Cu m stări și o vecinătate de k celule (inclusiv celula însăși) există m^k configurații locale și m^(m^k) reguli posibile – pentru m = 2, k = 3, adică 8 configurații și 256 de reguli.

Un exemplu din cursul profesorului Cernazean este **regula parității**: stări 0/1, vecinătate von Neumann, iar starea nouă este suma celor 4 vecini modulo 2. Pornind de la un model de 12 celule, apar figuri tot mai complicate, dar la anumiți pași (de exemplu 32) configurația este formată din **copii** exacte ale modelului

inițial: 48 = 4 × 12 celule vii. Regula fiind liniară (XOR), orice model se autoreproduce – o ilustrare simplă a ideii care l-a preocupat pe von Neumann.

1.2. Istoric: de la autoreproducere la Game of Life

Automatele celulare au apărut în anii 1940, din discuțiile dintre John von Neumann și Stanisław Ulam la Los Alamos. Von Neumann căuta logica unui mecanism care se poate **reproduce singur** – o întrebare pusă înaintea descoperirii structurii ADN-ului. A construit un automat cu 29 de stări în care o configurație de zeci de mii de celule își copiază propria descriere și construiește o copie a sa; automatul era, în plus, un calculator universal. În 1984, Christopher Langton a simplificat construcția la o **buclă** cu 8 stări, care nu mai este universală, dar conține propria „rețetă de fabricație" și un mecanism de citire și transformare a ei. Concluzia lui Cernazean: autoreproducerea poate fi privită mecanicist și nu cere o ierarhie în care ceva mai complicat construiește ceva mai simplu.

În 1970, John Conway a inventat **Game of Life**, făcut celebru de rubrica lui Martin Gardner din Scientific American. În anii 1980, Stephen Wolfram a studiat sistematic automatele 1D și le-a clasificat după comportament, iar Toffoli și Margolus au construit la MIT mașini hardware dedicate (CAM). Ideea de a modela fizica prin reguli locale discrete a dus la gazele pe rețea (HPP, 1973; FHP, 1986) și la metoda Lattice Boltzmann, folosită astăzi industrial.

1.3. Automatele elementare și clasele lui Wolfram

Un **automat elementar** este 1D, are 2 stări și vecinătatea formată din celulă și cei doi vecini. Cele 8 configurații locale se scriu în ordinea 111, 110, …, 000, iar biții stărilor noi, citiți ca număr binar, dau **numărul regulii** (Wolfram, 1983). Regula 30 are biții 00011110, regula 90 are biții 01011010 (stânga XOR dreapta), regula 110 are biții 01101110, iar regula 184 are biții 10111000. Evoluția se desenează ca o **diagramă spațiu–timp**: fiecare rând este un pas, de sus în jos (Figura 1).

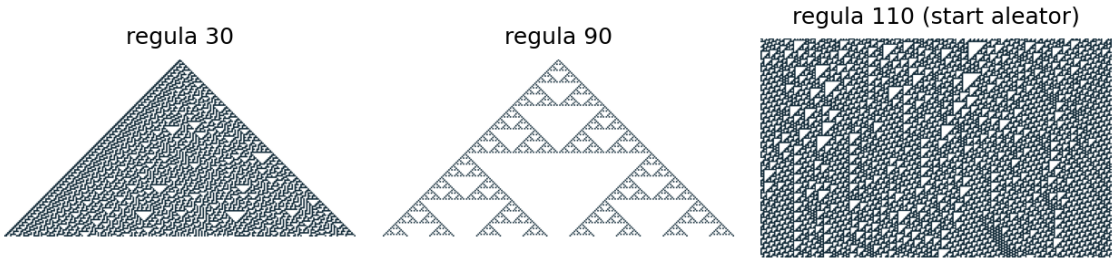


Figura 1. Automate elementare: regula 30 și regula 90 dintr-o singură celulă, regula 110 dintr-o stare inițială aleatoare

```
def eca(rule, width=201, steps=100):
    table = [(rule >> k) & 1 for k in range(8)]   # bitul k = config. k
    x = np.zeros(width, np.uint8); x[width // 2] = 1
    rows = [x]
    for _ in range(steps):
        idx = 4 * np.roll(x, 1) + 2 * x + np.roll(x, -1)
        x = np.array(table, np.uint8)[idx]
        rows.append(x)
    return np.array(rows)
```

Wolfram a propus patru **clase de comportament**: clasa I ajunge la o configurație uniformă (regula 32), clasa II la structuri stabile sau periodice (regula 108), clasa III produce modele aperiodice, aparent aleatoare (regulile 30 și 90), iar clasa IV produce structuri persistente care se deplasează și interacționează (regula 110, Game of Life). Regula 30 a fost folosită ca generator de numere aleatoare: primii biți ai coloanei centrale sunt 1101110011000101100100111010111O, cu o proporție de 1 de 51,9%. Regula 110 este un calculator **universal** (Cook, 2004), ceea ce face nedecidabile anumite întrebări despre evoluția ei. Regula 184 conservă numărul de 1 și poate fi citită ca un model de trafic.

1.4. Game of Life

Game of Life folosește grila pătrată, stări vii/moarte și vecinătatea Moore. Regula se notează **B3/S23**: o celulă moartă cu exact 3 vecini vii se naște, o celulă vie cu 2 sau 3 vecini vii supraviețuiește, iar în rest celula moare. Din aceste reguli apar structuri stabile (blocul), oscilatoare (intermitentul), nave care se deplasează (**planorul**, cu perioada 4 și viteza c/4) și chiar tunuri care emit planoare (Gosper, 1970). Planoarele pot fi folosite ca semnale, iar coliziunile lor ca porți logice: Life este un calculator universal (Berlekamp, Conway și Guy, 1982), în care s-a construit efectiv o mașină Turing (Rendell, 2000).

```
def life_step(a):
    # numărul de vecini vii (Moore, periodic)
    n = sum(np.roll(np.roll(a, i, 0), j, 1)
            for i in (-1, 0, 1) for j in (-1, 0, 1)
            if (i, j) != (0, 0))
    return ((n == 3) | ((a == 1) & (n == 2))).astype(np.uint8)
```

Implementarea vectorizată calculează numărul de vecini adunând 8 copii deplasate ale grilei și aplică regula ca o singură expresie booleană. Pe o grilă de 256 × 256, un pas durează 150 ms cu bucle Python și 0,27 ms cu NumPy – de aproximativ 557 de ori mai rapid, cu rezultat identic. Pornind de la o grilă de 200 × 200 cu 30% celule vii (11 946), după 1 000 de pași rămân 1 584 celule, organizate în structuri stabile și oscilante (Figura 2).

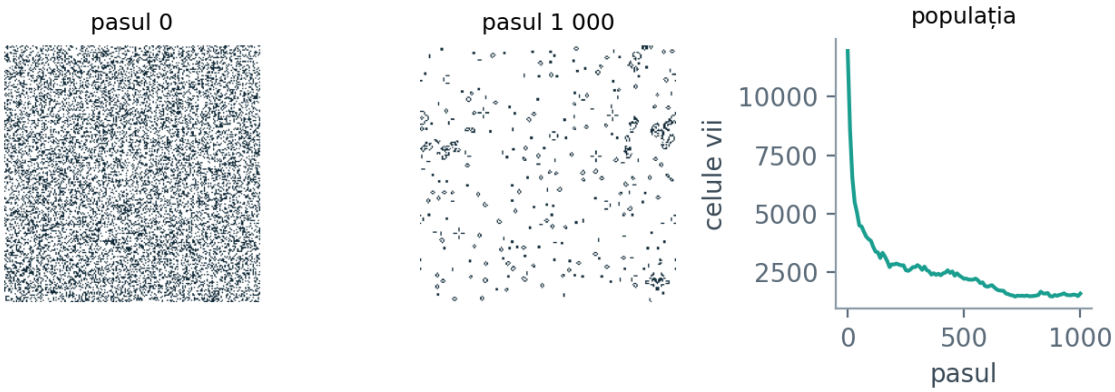


Figura 2. Game of Life pe 200 × 200 celule: starea inițială, starea după 1 000 de pași și numărul de celule vii

1.5. Emergență, ireductibilitate, variante

Furnica lui Langton se mișcă pe o grilă: pe o celulă albă se întoarce la dreapta, pe una neagră la stânga, schimbă culoarea celulei și înaintează. Aproximativ 10 000 de pași construiește o pată aparent haotică, apoi începe brusc o „autostradă” periodică (perioada 104) care pleacă la infinit (Figura 3). Lecția metodologică subliniată de Cernazean: cunoaștem perfect legea microscopică – noi am definit-o – și totuși nu putem prezice detaliile comportamentului macroscopic; singura cale este să observăm, adică să simulăm.

Proprietățile întregului care nu apar în regula unei singure celule se numesc **emergente**, iar sistemele pentru care nu există scurtături de calcul sunt **ireductibile computațional**.

Modelul **Greenberg–Hastings** descrie medii excitabile (țesut cardiac, neuroni): o celulă în repaus devine excitată dacă un vecin este excitat, apoi trece prin câteva stări refractare. Apar unde și spirale care se autoîntrețin, asemănătoare celor asociate aritmiilor cardiace. Definiția clasică are multe variante: automate **stocastice** (regula se aplică cu o probabilitate), **asincrone** (celulele se actualizează pe rând – evită oscilațiile artificiale, dar pierd paralelismul), **neuniforme**, cu stări continue (Lenia, 2019), **neurale** (Growing Neural CA, 2020, în care regula este învățată de o rețea neuronală mică) sau pe grafuri.

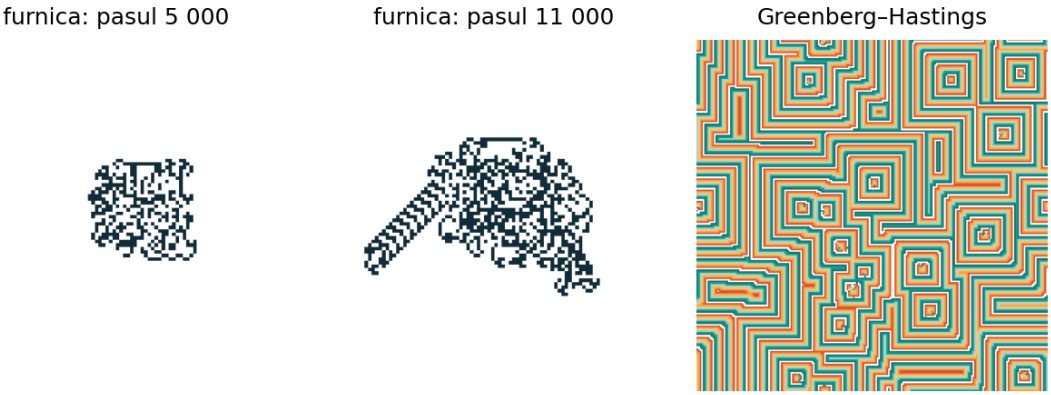


Figura 3. Furnica lui Langton la pașii 5 000 și 11 000 (începutul „autostrăzii") și un mediu excitabil Greenberg–Hastings

De reținut

Automat celular ⟨A, S, N, Φ⟩: rețea, stări, vecinătate (von Neumann, Moore), regulă locală, actualizare sincronă, condiții la margine (de obicei periodice). 256 de reguli elementare; clasele Wolfram I–IV; 30 (haos), 90 (Sierpiński), 110 (universală), 184 (trafic). Game of Life B3/S23: planoare, universalitate. Vectorizarea cu np.roll: de sute de ori mai rapidă. Emergență și ireductibilitate: uneori singura cale este simularea.

2. Automatele celulare ca modele ale realității

2.1. Ce face un model CA să fie bun

Cernazean pornește de la observația că realitatea are mai multe niveluri – microscopic, mezoscopic, macroscopic – și că **comportamentul macroscopic depinde puțin de detaliile microscopice**: supraviețuiesc doar simetriile și legile de conservare. Strategia este să construim o lume fictivă, simplă și ușor de simulat în paralel, care are același comportament macroscopic. Un model CA bun alege nivelul potrivit (ce reprezintă o celulă), păstrează legile de conservare (mașinile nu dispar; masa și impulsul unui fluid se conservă) și simetriile relevante, introduce aleatorul acolo unde detaliile nu contează și se validează pe mărimi macroscopice.

2.2. Traficul: regula 184 și modelul Nagel–Schreckenberg

Regula 184 este deja un model de trafic: o mașină (1) înaintează dacă celula din față este liberă. Pe un inel cu 1 000 de celule, sub densitatea 0,5 toate mașinile se mișcă și fluxul este J = ρ; peste 0,5 se mișcă doar mașinile

din fața golurilor și J = 1 − ρ. Graficul flux–densitate, numit **diagrama fundamentală**, are forma unui cort cu vârful 0,5 la densitatea 0,5 (Figura 5, stânga).

Modelul Nagel–Schreckenberg (1992) adaugă viteze întregi $v \in \{0, \ldots, v_{max}\}$, cu $v_{max} = 5$ (o celulă ≈ 7,5 m, un pas ≈ 1 s, deci viteza maximă ≈ 135 km/h). La fiecare pas, pentru toate mașinile simultan: (1) **accelerare** $v \leftarrow \min(v + 1, v_{max})$; (2) **frânare** $v \leftarrow \min(v, d)$, unde d este numărul de celule libere până la mașina din față; (3) **încetinire aleatoare**: cu probabilitatea p, $v \leftarrow \max(v - 1, 0)$; (4) **deplasare** cu v celule. Implementarea stochează pozițiile și vitezele mașinilor, nu celulele drumului.

```
def nasch_step(pos, v):
    gap = (np.roll(pos, -1) - pos - 1) % L
    v = np.minimum(v + 1, vmax)          # 1. accelerare
    v = np.minimum(v, gap)               # 2. frânare
    slow = (rng.random(N) < p) & (v > 0)
    v = np.where(slow, v - 1, v)         # 3. încetinire aleatoare
    pos = (pos + v) % L                  # 4. deplasare
    k = np.argsort(pos)
    return pos[k], v[k]                  # flux: J = v.sum() / L
```

Fără încetinirea aleatoare (p = 0), după o perioadă de rodaj traficul devine uniform. Cu p = 0,25 apar **ambuteiaje spontane** care se propagă înapoi, în sens opus circulației (Figura 4) – exact ce se observă pe autostrăzi, fără accident sau lucrări. Experimentul lui Sugiyama și colaboratorii (2008), cu 22 de mașini pe un circuit de 230 m, a reprodus controlat acest fenomen: la aproximativ 30 km/h, după câteva minute apare un ambuteiaj care se deplasează înapoi. Studii mai recente arată că o singură mașină autonomă bine controlată poate amortiza aceste unde (Stern et al., 2018).

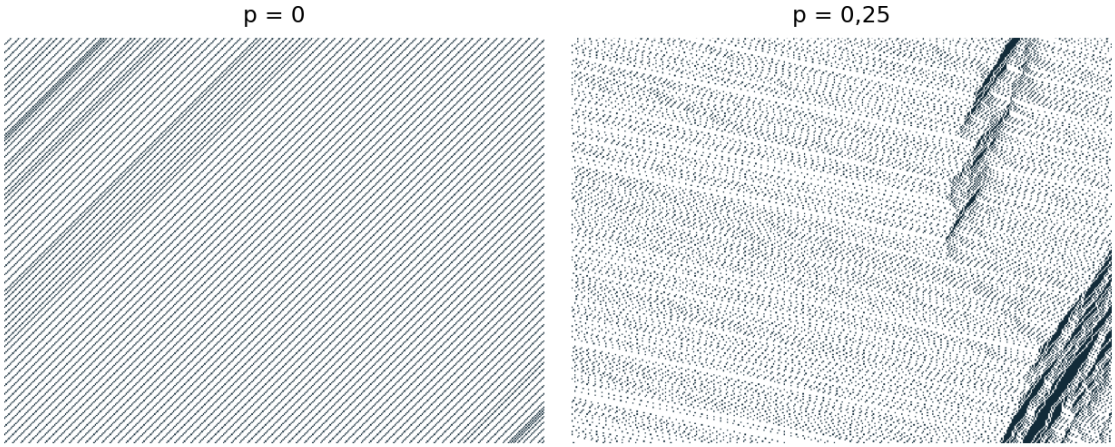


Figura 4. Diagrame spațiu–timp NaSch la densitatea 0,18 (300 de pași, timpul în jos): fără și cu încetiniri aleatoare

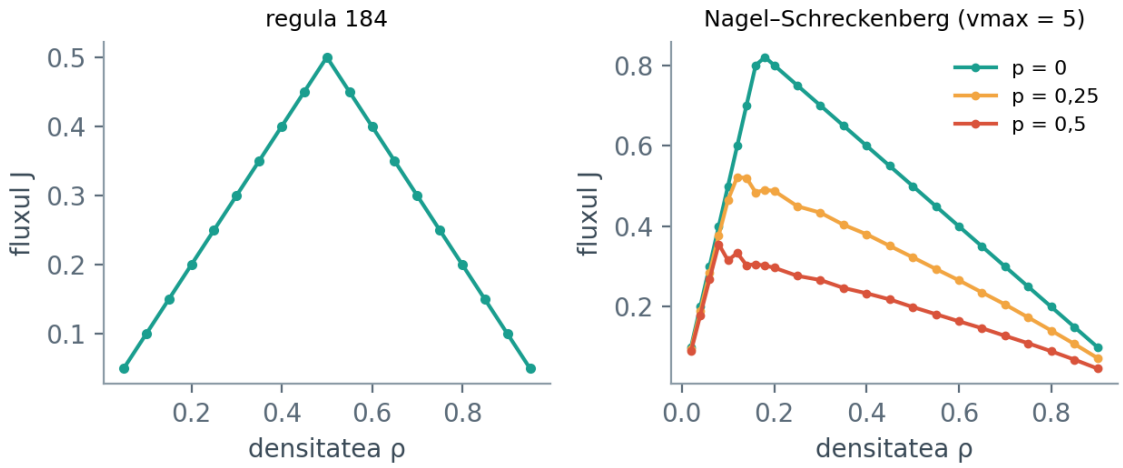


Figura 5. Diagrama fundamentală pentru regula 184 și pentru modelul Nagel–Schreckenberg cu trei valori ale lui p

Tabelul 1. Capacitatea drumului în modelul NaSch (inel de 1 000 de celule, $v_{max} = 5$)

Probabilitatea p	Flux maxim	Densitatea critică	Flux la ρ = 0,5
0,00	0,820	0,18	0,500
0,25	0,522	0,12	0,323
0,50	0,356	0,08	0,200

Încetinirile aleatoare reduc capacitatea drumului: fluxul maxim scade de la 0,82 la 0,52 (circa 36% mai puține mașini pe oră) și apare la densități mai mici. Concluzia practică este că tot ce omogenizează vitezele – limite variabile, pilot automat adaptiv – crește capacitatea. Modele de acest tip au fost folosite la scară mare: TRANSIMS (Los Alamos, anii 1990), prognoza OLSIM pentru autostrăzile din Renania de Nord-Westfalia și modelul rețelei din Geneva prezentat de Cernazean (1 066 de intersecții, 3 145 de segmente, 560 886 de celule, 85 055 de mașini). Aceleași idei se aplică pietonilor (modelul floor-field, Burstedde et al., 2001).

2.3. Incendii și percolare

În modelul incendiului, fiecare celulă a unei păduri pătrate conține un copac cu probabilitatea p, focul pornește de pe marginea din stânga, iar un copac arde dacă un vecin (von Neumann) arde. Pe o pădure de 100 × 100, la p = 0,55 arde doar 5,3% din copaci, iar la p = 0,65 arde 92,3% și focul traversează pădurea (Figura 6). Repetând pe 200 de păduri, probabilitatea de traversare trece de la 19,5% la p = 0,58 la 94,5% la p = 0,62: un salt brusc în jurul **pragului de percolare** pc ≈ 0,5927 al rețelei pătrate.

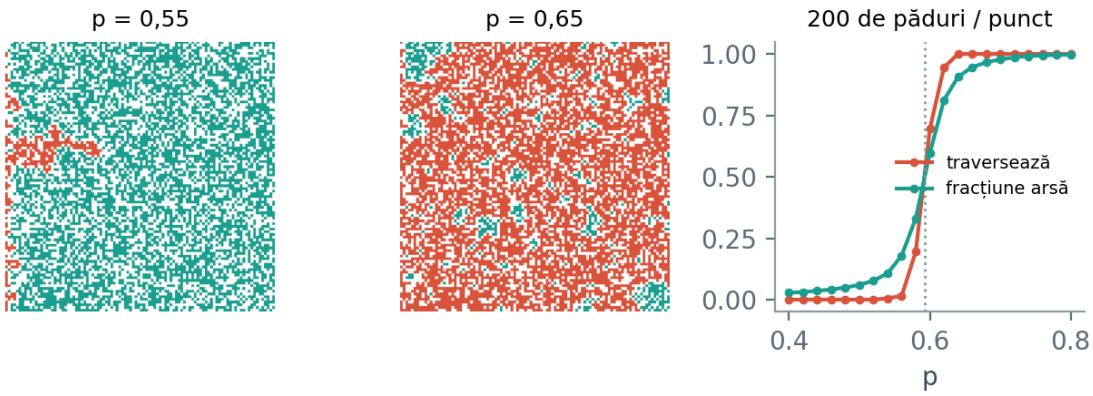


Figura 6. Incendiul de pădure (verde: neatins, roșu: ars) și probabilitatea de traversare în funcție de densitate; linia punctată marchează pc ≈ 0,5927

Percolarea este una dintre cele mai simple **tranziții de fază**: sub prag, grupurile de copaci sunt finite; peste prag apare un grup care traversează sistemul. Aceeași matematică descrie apa care trece prin roci poroase sau prin cafea, curentul în materiale compozite și epidemiile pe rețele de contacte. Pentru simulare nu este necesar să propagăm focul pas cu pas: este suficientă etichetarea componentelor conexe (scipy.ndimage.label). Varianta dinamică Drossel–Schwabl (1992), cu copaci care cresc și fulgere, este un exemplu de criticalitate autoorganizată.

2.4. Fulgi de zăpadă, diferențiere celulară, difuzie

Pe o rețea **hexagonală**, regula „o celulă îngheață dacă exact un vecin este înghețat” produce, pornind de la o singură celulă, forme cu simetrie de ordin 6 și ramificații asemănătoare fulgilor reali (după 55 de pași: 5 323 de celule înghețate; Packard, 1984). În biologie, Cernazean prezintă diferențierea celulelor în embrionul de Drosophila: celulele, inițial identice, devin în proporție de puțin sub 25% neuroblaste; ipoteza este că fiecare celulă produce o substanță care declanșează diferențierea la un prag, iar vecinii se inhibă reciproc. Un automat cu inhibiție laterală reproduce proporția observată.

Difuzia se poate scrie ca automat celular cu stări reale: u(t + 1) = u(t) + α·(suma celor 4 vecini − 4u), cu α = DΔt/Δx². Este exact discretizarea explicită a ecuației căldurii, iar masa totală se conservă (400 la început și 400 după 1 000 de pași). Schema este stabilă doar pentru α ≤ 1/4; cu α = 0,3, după 60 de pași valorile ajung la ±900 000 – aceeași lecție ca la metoda Euler cu pas prea mare (S7).

2.5. Gaze pe rețea și metoda Lattice Boltzmann

Pentru fluide, Cernazean insistă: „conservarea exactă a masei și impulsului – asta contează cu adevărat”. Modelul **HPP** (Hardy, de Pazzis, Pomeau, 1973) folosește o rețea pătrată cu 4 direcții și cel mult o particulă pe direcție; la o coliziune frontală particulele se rotesc cu 90°. Masa și impulsul se conservă, dar rețeaua pătrată nu este destul de simetrică și curgerea este anizotropă. Modelul **FHP** (Frisch, Hasslacher, Pomeau, 1986) trece la o rețea hexagonală cu 6 direcții și reproduce, la scară mare, ecuațiile Navier–Stokes. Microdinamica se scrie $n_i(r + c_i, t + 1) = n_i(r, t) + \Omega_i(n)$: propagare pe direcția i și coliziune locală Ω_i care conservă numărul și impulsul particulelor.

Gazele booleene au zgomot statistic mare. **Metoda Lattice Boltzmann** (McNamara și Zanetti, 1988) înlocuiește $n_i \in \{0, 1\}$ cu media $f_i \in [0; 1]$. Cu operatorul de coliziune BGK: $f_i(x + c_i, t + 1) = f_i - \omega(f_i - f_i(eq))$, unde echilibrul $f_i(eq) = w_i\rho(1 + 3c_i{\cdot}u + 4{,}5(c_i{\cdot}u)^2 - 1{,}5u^2)$ depinde de densitatea $\rho = \Sigma f_i$ și de viteza u, cu $\rho u = \Sigma f_i c_i$. Pe rețeaua **D2Q9** ponderile sunt 4/9 (repaus), 1/9 (axe) și 1/36 (diagonale), viteza sunetului pe rețea are

$cs^2 = 1/3$, iar vâscozitatea este $\nu = cs^2(1/\omega - 1/2)$, deci $0 < \omega < 2$. Fiecare pas are o etapă de coliziune (locală) și una de propagare (deplasări de vectori), ambele perfect paralele.

```
for it in range(steps):
    rho = f.sum(axis=0)
    u = np.einsum('id,ixy->dxy', c, f) / rho
    feq = equilibrium(rho, u)
    fout = f - omega * (f - feq)            # coliziune BGK
    for i in range(9):                      # bounce-back pe obstacol
        fout[i, obst] = f[8 - i, obst]
    for i in range(9):                      # propagare
        f[i] = np.roll(fout[i], c[i], axis=(0, 1))
```

Simularea curgerii în jurul unui cilindru (grilă 400 × 100, Re = 150, $\nu = 0{,}00293$, $\omega = 1{,}9654$) produce, după 14 000 de pași, **aleea de vârtejuri von Kármán** (Figura 7). Calculul a durat ~108 s pe un singur nucleu cu NumPy (≈ 5,2 milioane de actualizări de noduri pe secundă); un solver C++ optimizat sau un GPU este de sute până la mii de ori mai rapid. Niciun termen din ecuațiile Navier–Stokes nu apare explicit în cod: echivalența la scară mare se demonstrează prin analiza Chapman–Enskog, pentru viteze mici față de viteza sunetului pe rețea. Biblioteca Palabos (Geneva), prezentată de Cernazean, implementează LBM paralel, cu domeniul împărțit în blocuri.

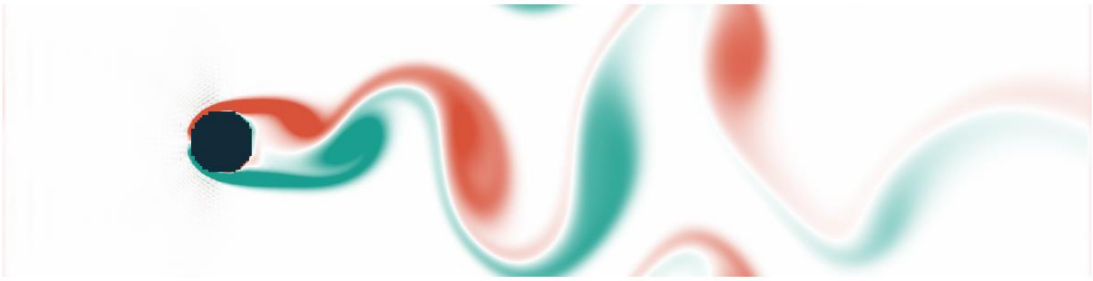


Figura 7. Vorticitatea în curgerea în jurul unui cilindru, simulată cu Lattice Boltzmann D2Q9 (Re = 150)

2.6. CA sau EDP; aplicații în informatică; limite

Automatele celulare sunt potrivite când fenomenul este discret prin natura lui (mașini, copaci, indivizi), când cunoaștem regulile locale, dar nu ecuația macroscopică, când geometria este complicată sau când vrem paralelism masiv și simplu. Ecuațiile cu derivate parțiale sunt preferabile când avem o ecuație bine validată și avem nevoie de precizie controlată și de analiză de erori. Granița este fluidă: difuzia explicită este simultan EDP discretizată și CA, iar LBM este folosit ca solver Navier–Stokes.

În informatică, automatele celulare apar în generarea procedurală a hărților din jocuri (o grilă aleatoare cu 45% pereți, netezită de 5 ori cu regula majorității, devine un sistem de peșteri), în procesarea imaginilor (eroziune, dilatare, filtrul majorității), în generatoare de numere aleatoare (regula 30, automate hibride 30/150), în Wireworld (circuite logice) și în simulările „fiecare pixel este o celulă” din jocuri. Limitele lor sunt anizotropia rețelei, artefactele actualizării sincrone, calibrarea regulilor, alegerea rezoluției și memoria necesară în 3D.

De reținut

Model CA bun: nivel potrivit, legi de conservare, simetrii, aleator unde e cazul, validare macroscopică.
NaSch: accelerare, frânare, încetinire aleatoare (p), deplasare; încetinirea aleatoare produce

ambuteiaje fantomă și reduce capacitatea. Percolare: tranziție bruscă la pc ≈ 0,593 (rețea pătrată). Difuzie explicită: stabilă pentru α ≤ 1/4, conservă masa. LBM: coliziune BGK + propagare; D2Q9 (4/9, 1/9, 1/36); v = (1/ω − 1/2)/3.

3. Particule și simulare de înaltă performanță

3.1. Ecuațiile de mișcare și potențialele

Într-un sistem de particule urmărim obiectele care se mișcă (descriere lagrangiană), nu puncte fixe ale spațiului (descriere euleriană). Obiectele – stele, molecule, globule roșii, boabe de nisip – sunt simplificate la puncte cu poziție, viteză și câteva proprietăți, tocmai pentru a putea simula un număr uriaș (Cernazean: aproximativ un miliard de globule roșii într-o arteră). Mișcarea respectă legea a doua a lui Newton, $m_i \cdot d^2 r_i/dt^2 = F_i = \Sigma_j F(r_i, r_j)$, cu forțe pe perechi derivate dintr-un potențial ($F = -\nabla V$). Datorită legii a treia ($F_{ij} = -F_{ji}$), fiecare pereche se calculează o singură dată: $N(N-1)/2$ perechi.

Tabelul 2. Potențiale de interacțiune frecvente

Potențial	V(r)	Rază de acțiune	Folosit pentru
Gravitație	$-G \cdot m_1 m_2 / r$	lungă	stele, galaxii
Coulomb	$k \cdot q_1 q_2 / r$	lungă	ioni, plasme
Lennard-Jones	$4\varepsilon[(\sigma/r)^{12} - (\sigma/r)^6]$	scurtă (tăiere la 2,5σ)	gaze nobile, lichide
Resort (Hooke)	$k(r - r_0)^2 / 2$	perechi legate	legături, haine
Contact (Hertz)	$\propto \delta^{5/2}$	doar la atingere	materiale granulare

Potențialul **Lennard-Jones** combină o respingere puternică la distanțe mici (r^{-12}) cu o atracție slabă (r^{-6}, forțe van der Waals). Minimul $V = -\varepsilon$ apare la $r = 2^{1/6}\sigma \approx 1{,}1225\sigma$. La 2,5σ potențialul este -0,0163ε, adică 1,6% din adâncimea minimului (în materialul original apare „aproximativ 1%"); de aceea interacțiunile mai îndepărtate se neglijează (**rază de tăiere**). Potențialele cu rază lungă (gravitație, Coulomb) nu permit tăierea și cer alți algoritmi.

3.2. Integrarea: metoda Verlet

Algoritmul **Verlet** (1967) se obține adunând seriile Taylor pentru x(t + Δt) și x(t − Δt): $x(t + \Delta t) = 2x(t) - x(t - \Delta t) + a(t) \cdot \Delta t^2$. Termenii cu viteza și cu Δt^3 se anulează, eroarea locală pentru poziții este $O(\Delta t^4)$, iar pe pas se face o singură evaluare a forțelor. (În slide-urile originale apare o greșeală de tipar, „xi2(t)" în loc de $2x_i(t)$.) Varianta **velocity Verlet** păstrează pozițiile și vitezele la aceleași momente, cu trei etape: $v \leftarrow v + a \cdot \Delta t/2$; $x \leftarrow x + v \cdot \Delta t$; $v \leftarrow v + a(x) \cdot \Delta t/2$. Varianta **leapfrog** păstrează vitezele la jumătăți de pas; toate trei produc aceleași traiectorii.

```
def velocity_verlet(x, v, acc, dt, steps):
    a = acc(x)
    for _ in range(steps):
        v_half = v + 0.5 * dt * a
        x = x + dt * v_half
        a = acc(x)                    # o singură evaluare pe pas
        v = v_half + 0.5 * dt * a
    return x, v
```

Comparăm trei metode pe orbita circulară a unei planete (GM = 1, r = 1, energie exactă −0,5), cu Δt = 0,05, pe 100 de orbite (Figura 8). Euler adaugă energie la fiecare pas: eroarea ajunge la 0,416, iar planeta aproape evadează. RK4 are o eroare mică, dar care crește liniar ($2{,}71\cdot10^{-6}$ pe orbita 100), în timp ce eroarea Verlet rămâne aceeași pe fiecare orbită ($7{,}78\cdot10^{-7}$) – deja la 100 de orbite mai mică decât a lui RK4, deși Verlet costă de 4 ori mai puțin.

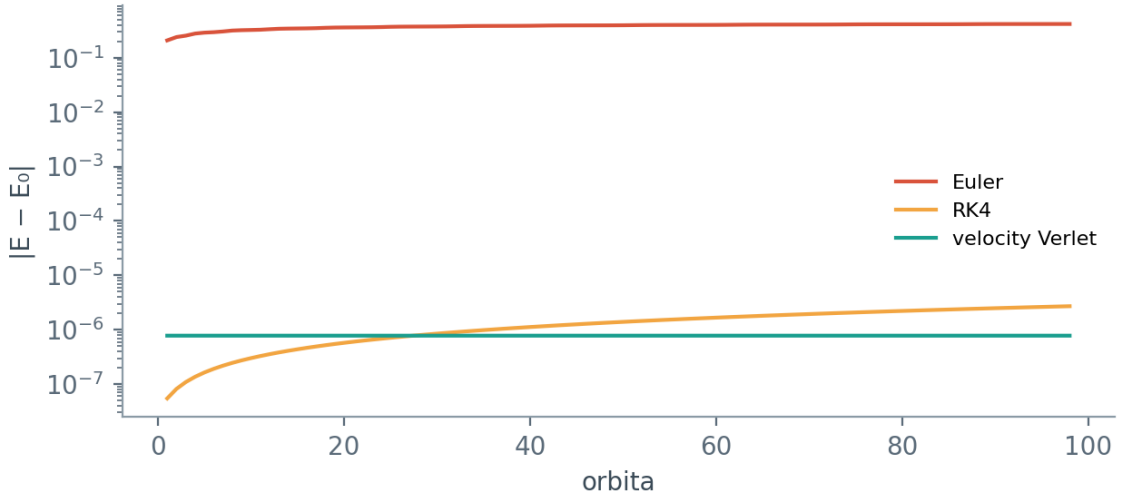


Figura 8. Eroarea maximă a energiei pe fiecare orbită, pentru Euler, RK4 și velocity Verlet (Δt = 0,05; scară logaritmică)

Tabelul 3. Euler, RK4 și Verlet pe orbita Kepler

Metoda	Forțe pe pas	Eroarea pe orbita 100	Comportament
Euler	1	0,416	crește: orbita se lărgește
RK4	4	$2{,}71\cdot10^{-6}$	derivă lentă, liniară
velocity Verlet	1	$7{,}78\cdot10^{-7}$	mărginită, nu se acumulează

Verlet este standardul în dinamica moleculară pentru că este **simplectică** (păstrează volumul în spațiul fazelor și conservă exact o energie „vecină”, deci eroarea energiei rămâne mărginită), **reversibilă în timp**, ieftină (o evaluare a forțelor pe pas) și are nevoie de puțină memorie. În simulări cu 10^6–10^9 pași, păstrarea structurii sistemului contează mai mult decât precizia unui singur pas. Legătura cu S7: Euler simplectic păstra orbitele Lotka–Volterra; Verlet este varianta lui de ordinul 2.

3.3. Costul calculului forțelor: grilă, arbori, Barnes–Hut

Cu un milion de particule există ~$5\cdot10^{11}$ perechi; la ~10^9 perechi pe secundă, un singur pas ar dura 8 minute. Pentru forțe cu rază scurtă folosim **metoda grilei** (cell list): spațiul se împarte în celule cu latura cel puțin egală cu raza de tăiere, fiecare particulă se înscrie în celula ei, iar vecinii unei particule pot fi doar în cele 9 celule din jur (27 în 3D). La densitate constantă, costul devine O(N). În practică se folosesc structuri optimizate, cum sunt arborii k-d (scipy.spatial.cKDTree) sau listele Verlet de vecini.

Măsurătorile noastre (densitate 0,5, rază 2,5) arată diferența (Figura 9, stânga): la N = 8 000, calculul tuturor distanțelor (vectorizat) durează 2 453 ms, iar arborele k-d 6,03 ms; când dublăm N, prima metodă devine de 4 ori mai lentă, a doua de 2 ori. Ambele găsesc exact aceleași 38 646 de perechi.

Gravitația nu are rază de tăiere. **Algoritmul Barnes–Hut** (1986) împarte recursiv spațiul în pătrate (quadtree; octree în 3D) până când fiecare celulă conține cel mult un corp, iar fiecare nod reține masa totală și centrul de masă. Pentru forța asupra unui corp parcurgem arborele: dacă $s/d < \theta$ (s – latura celulei, d – distanța până la centrul ei de masă), tratăm celula ca un singur corp; altfel coborâm în fii. Cu $\theta = 0{,}5$, numărul de interacțiuni pe corp crește doar logaritmic: 56,8 la $N = 250$ și 131,5 la $N = 4\,000$, față de 3 999 la calculul direct, cu o eroare mediană a forței de 0,6–1,2% (Figura 9, dreapta).

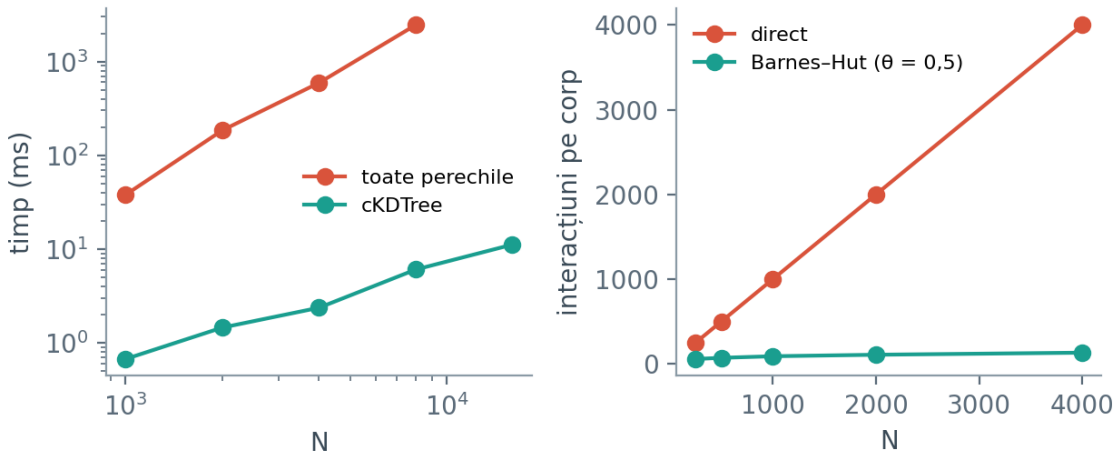


Figura 9. Stânga: timpul de găsire a perechilor apropiate, toate perechile vs arbore k-d (scară log–log). Dreapta: interacțiuni pe corp, direct vs Barnes–Hut

Tabelul 4. Complexitatea calculului forțelor

Metoda	Cost pe pas	Când se folosește
Toate perechile	$O(N^2)$	N mic, cod de referință, verificare
Rază de tăiere + grilă	$O(N)$	forțe cu rază scurtă (LJ, DEM, SPH)
Barnes–Hut	$O(N \log N)$	gravitație, N mare
Multipoli rapizi (FMM)	$O(N)$	gravitație, Coulomb, precizie mare
Particle-mesh (PME)	$O(N \log N)$	electrostatică periodică (FFT)

Aceleași idei apar în aplicații foarte diferite: simulări cosmologice cu zeci de miliarde de particule (Millennium, 2005), dinamică moleculară pe hardware dedicat (Anton), fluide SPH, metoda elementelor discrete pentru materiale granulare și motoarele fizice din jocuri, unde grilele și arborii elimină rapid perechile îndepărtate („broad phase") înainte de calculul contactelor. Un algoritm mai bun câștigă ordine de mărime; un hardware mai rapid câștigă doar factori constanți.

3.4. Performanță: vectorizare, paralelism, GPU

Un model bun trebuie să fie precis și să dea rezultatul într-un timp rezonabil. Cernazean compară mai multe implementări ale sumei a 10^6 numere: același algoritm diferă de sute de ori ca timp, doar din implementare și din ordinea accesului la memorie (Tabelul 5).

Tabelul 5. Suma a 10^6 numere: măsurătorile noastre și cele din Cernazean L2

Implementarea	Noi (ms)	Raport față de NumPy	Cernazean (ms)
Python, citire liniară	33,6	124×	59,4
Python, citire aleatoare	184	681×	270
Python, sum()	3,61	13×	6,54
NumPy, a.sum()	0,27	1×	0,634
C++, buclă liniară	–	–	0,478

NumPy este rapid pentru că bucla rulează în cod compilat, pe date așezate contiguu în memorie, folosind cache-ul și instrucțiunile SIMD. Accesul aleator ratează cache-ul: o citire din memoria principală costă de ordinul a 100 ns, față de ~1 ns din cache. Concluzia lui Cernazean: scris naiv, C++ este de aproximativ 100 de ori mai rapid decât Python, dar cu NumPy performanța devine comparabilă.

Paralelismul are două modele. În **memoria partajată** (OpenMP, fire de execuție), mai multe nuclee ale aceluiași calculator lucrează pe aceeași memorie; paralelizarea este simplă, dar trebuie evitate condițiile de cursă. În **memoria distribuită** (MPI), fiecare calculator are memoria lui și datele se trimit explicit prin mesaje; modelul scalează la milioane de nuclee, cu prețul comunicării. Pentru grile se folosește **descompunerea domeniului**: fiecare procesor primește un bloc și schimbă cu vecinii doar marginile (halo). Pentru un bloc n × n, calculul crește cu n^2, iar comunicarea cu 4n: la blocuri 10 × 10 marginea reprezintă 40% din celule, la 1 000 × 1 000 doar 0,4%. Palabos folosește exact această abordare „multi-block", eficientă dacă comunicarea este mult mai mică decât calculul și încărcarea procesoarelor este echilibrată.

Legea lui Amdahl (1967) dă accelerarea cu N procesoare când doar fracțiunea p a programului se poate paraleliza: S(N) = 1/((1 − p) + p/N). Pentru N foarte mare, S tinde la 1/(1 − p): cu p = 0,9, accelerarea nu poate depăși 10 (9,91 la 1 024 de procesoare), iar cu p = 0,99 nu poate depăși 100 (91,18 la 1 024; Figura 10). Legea lui Gustafson (1988) este mai optimistă: dacă mărim problema odată cu numărul de procesoare, partea paralelă crește și ea. Se paralelizează ușor celulele unui CA la același pas, forțele pe particule diferite și replicările independente Monte Carlo; greu sau deloc recurențele de tip Fibonacci, pașii de timp succesivi, lista de evenimente a unei simulări discrete și actualizarea asincronă.

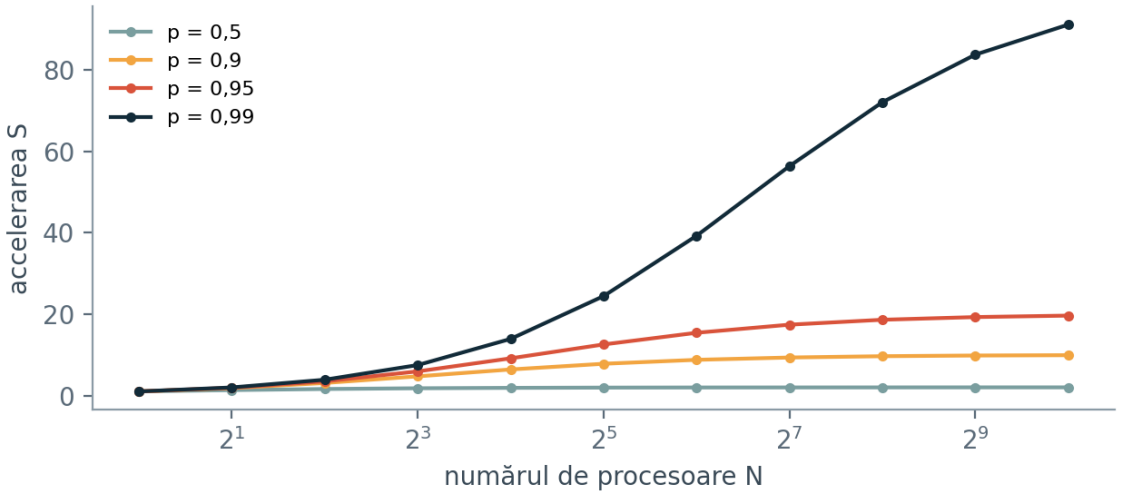


Figura 10. Accelerarea după legea lui Amdahl pentru patru fracțiuni paralelizabile p

Procesoarele grafice au mii de nuclee simple care rulează același program pe date diferite: Cernazean menționează ~2–3 TFLOPS pentru un procesor modern și ~80 TFLOPS pentru un GPU de consum. Sunt ideale pentru calcule regulate (CA, LBM, perechi de particule, rețele neuronale) și se pot folosi din Python prin CuPy, JAX, Numba sau PyTorch; datele trebuie să rămână pe GPU, pentru că transferul CPU–GPU este scump. La vârf, lista TOP500 din iunie 2026 are pe primul loc sistemul LineShine (Shenzhen, China) cu 2,198 EFlop/s, construit doar cu procesoare CPU, urmat de El Capitan (1,809), Frontier (1,353) și Aurora (1,012); JUPITER (Jülich, Germania) a devenit în noiembrie 2025 primul sistem exascale al Europei. Există 5 sisteme exascale; cifra de „~1 EFlop/s" din cursul Cernazean (2024) a fost depășită. Ordinea recomandată a optimizării rămâne: măsurare (profiler), algoritm, vectorizare, cod compilat, paralelism, GPU.

De reținut

Particule: Newton + potențiale pe perechi (LJ: minim la $2^{1/6}\sigma$, tăiere la $2{,}5\sigma$). Velocity Verlet: o evaluare de forțe pe pas, simplectică, eroare de energie mărginită. Cost: $O(N^2)$ direct; $O(N)$ cu rază de tăiere și grilă; $O(N \log N)$ cu Barnes–Hut ($s/d < \theta$). Performanță: NumPy ≈ C++ ≫ bucle Python; memorie partajată vs distribuită; descompunerea domeniului (halo ∝ perimetru); Amdahl: $S \le 1/(1-p)$; GPU pentru calcule regulate.

Glosar RO – EN

Română	English
automat celular	cellular automaton
rețea	lattice
vecinătate	neighbourhood
regulă locală	local rule
actualizare sincronă / asincronă	synchronous / asynchronous update
condiții la margine periodice	periodic boundary conditions
automat elementar	elementary cellular automaton
diagramă spațiu–timp	space–time diagram
planor	glider
emergență	emergence
ireductibilitate computațională	computational irreducibility
mediu excitabil	excitable medium
diagrama fundamentală	fundamental diagram
ambuteiaj fantomă	phantom traffic jam
prag de percolare	percolation threshold
tranziție de fază	phase transition

Română	English
gaz pe rețea	lattice gas
coliziune / propagare	collision / streaming
vâscozitate	viscosity
aleea de vârtejuri von Kármán	von Kármán vortex street
potențial	potential
rază de tăiere	cut-off radius
metoda grilei	cell list (grid method)
arbore quadtree / octree	quadtree / octree
centru de masă	centre of mass
metodă simplectică	symplectic method
vectorizare	vectorisation
memorie partajată / distribuită	shared / distributed memory
descompunerea domeniului	domain decomposition
accelerare	speed-up
legea lui Amdahl	Amdahl's law
supercalculator	supercomputer

Întrebări de autoevaluare

1. Ce înseamnă actualizarea **sincronă** a unui automat celular?

a) celulele se actualizează pe rând, într-o ordine aleatoare

b) toate celulele calculează starea nouă din stările de la pasul anterior, simultan

c) doar celulele vii se actualizează

d) fiecare celulă are propriul ceas

2. Câți vecini are o celulă în vecinătatea **Moore** pe o grilă pătrată 2D?

a) 4

b) 6

c) 8

d) 9

3. Regula elementară 90 poate fi scrisă ca:

a) starea nouă = stânga XOR dreapta

b) starea nouă = majoritatea celor trei celule

c) starea nouă = 1 dacă celula din față e liberă

d) starea nouă = NOT centru

4. Care automat este exemplul tipic pentru **clasa IV** Wolfram (structuri persistente care interacționează, calcul universal)?

a) regula 32

b) regula 108

c) regula 30

d) regula 110

5. În Game of Life (B3/S23), o celulă **vie** are **4** vecini vii. Ce se întâmplă la pasul următor?

a) supraviețuiește

b) se multiplică

c) moare (suprapopulare)

d) rămâne vie doar dacă avea 3 vecini la pasul anterior

6. Ce lecție metodologică ilustrează furnica lui Langton?

a) orice sistem determinist este periodic

b) cunoașterea exactă a regulii microscopice nu permite întotdeauna prezicerea comportamentului macroscopic – trebuie să simulăm

c) automatele celulare nu pot produce structuri regulate

d) aleatorul este necesar pentru complexitate

7. În regula 184 ca model de trafic, care este fluxul de mașini (mașini care se mișcă pe celulă și pe pas) la densitatea **ρ = 0,7**, în regim staționar?

a) 0,3

b) 0,5

c) 0,7

d) 0,21

8. Ce efect are creșterea probabilității de încetinire aleatoare p în modelul Nagel–Schreckenberg?

a) crește fluxul maxim

b) elimină ambuteiajele

c) nu schimbă diagrama fundamentală

d) scade fluxul maxim și densitatea la care apare

9. În modelul incendiului pe o pădure pătrată mare, ce se întâmplă când densitatea copacilor trece de pragul de percolare $p_c \approx 0{,}593$?

a) focul se stinge mereu imediat

b) probabilitatea ca focul să traverseze pădurea crește brusc de la aproape 0 la aproape 1

c) fracțiunea arsă crește liniar cu p

d) nu se schimbă nimic calitativ

10. De ce modelul de gaz pe rețea **HPP** (rețea pătrată) nu reproduce corect un fluid, iar FHP (rețea hexagonală) da?

a) HPP nu conservă masa

b) HPP nu conservă impulsul

c) rețeaua pătrată nu este suficient de simetrică: curgerea simulată este anizotropă

d) HPP folosește stări reale în loc de booleene

11. Ce face regula **bounce-back** într-o simulare Lattice Boltzmann?

a) pe nodurile obstacolului, densitățile se întorc pe direcția opusă, ceea ce impune viteză nulă la perete

b) resetează simularea când devine instabilă

c) calculează echilibrul local

d) impune viteza la intrarea în canal

12. De ce putem folosi o **rază de tăiere** de 2,5σ pentru potențialul Lennard-Jones, dar nu pentru gravitație?

a) gravitația este mai slabă

b) Lennard-Jones nu are minim

c) gravitația se calculează doar în 2D

d) la 2,5σ potențialul LJ a scăzut la ~1,6% din adâncimea minimului, în timp ce gravitația scade lent (1/r) și se adună de la toate corpurile

13. De ce este preferată metoda **velocity Verlet** în simulările lungi de dinamică moleculară?

a) are ordinul 4, ca RK4

b) cere o singură evaluare a forțelor pe pas și, fiind simplectică, are eroarea energiei mărginită

c) folosește pas adaptiv

d) nu are nevoie de forțe

14. Care este costul pe pas al algoritmului **Barnes–Hut** pentru N corpuri gravitaționale?

a) $O(N^2)$

b) $O(N^3)$

c) $O(N \log N)$

d) $O(1)$

15. Care dintre aceste calcule este **cel mai greu** de paralelizat?

a) șirul lui Fibonacci, $F(i) = F(i-1) + F(i-2)$

b) actualizarea tuturor celulelor din Game of Life la același pas

c) forțele pe particule diferite, cu pozițiile cunoscute

d) 1 000 de replicări independente ale unei simulări Monte Carlo

16. Modelul Nagel–Schreckenberg, $v_{max} = 5$, fără încetinire aleatoare ($p = 0$). O mașină are viteza 4 și **2** celule libere până la mașina din față. Cu câte celule avansează la acest pas?

(Răspuns numeric)

17. Un automat celular 1D are **3** stări pe celulă și vecinătatea formată din celulă și cei doi vecini. Câte configurații locale posibile există?

(Răspuns numeric)

18. Difuzie 2D cu schema explicită, D = 1 și Δx = 0,1. Care este cel mai mare pas de timp Δt pentru care schema este stabilă ($\alpha = D\Delta t/\Delta x^2 \leq 1/4$)?

(Răspuns numeric)

19. Într-o simulare Lattice Boltzmann D2Q9 cu ω = 1,5, cât este vâscozitatea în unități de rețea, $\nu = (1/\omega - 1/2)/3$? (4 zecimale)

(Răspuns numeric)

20. Un program are **95%** din timp paralelizabil. Ce accelerare dă legea lui Amdahl pe **20** de procesoare? (2 zecimale)

(Răspuns numeric)

Răspunsuri

1. b) – În actualizarea sincronă, starea nouă a fiecărei celule depinde doar de stările vecinilor de la pasul anterior, iar toate celulele se schimbă în același timp.

2. c) – Vecinătatea Moore include cei 4 vecini de pe laturi și cei 4 de pe diagonale; vecinătatea von Neumann are doar 4.

3. a) – Regula 90 (01011010) dă 1 exact când unul singur dintre vecinii stâng și drept este 1; pornind de la o celulă produce triunghiul lui Sierpiński.

4. d) – Regula 110 are structuri localizate care se mișcă și interacționează; Cook a demonstrat în 2004 că este universală. Regula 30 este clasa III, 108 clasa II, 32 clasa I.

5. c) – O celulă vie supraviețuiește doar cu 2 sau 3 vecini vii; cu 4 sau mai mulți moare de suprapopulare.

6. b) – Regula furnicii este cunoscută perfect, dar apariția autostrăzii după ~10 000 de pași nu poate fi prezisă fără simulare: emergență și ireductibilitate computațională.

7. a) – Peste densitatea 0,5 se pot mișca doar mașinile din fața golurilor, deci fluxul este $1 - \rho = 0{,}3$. Sub 0,5, fluxul ar fi egal cu ρ.

8. d) – În simulările noastre, fluxul maxim scade de la 0,82 (p = 0) la 0,52 (p = 0,25) și 0,36 (p = 0,5), iar densitatea critică scade și ea.

9. b) – Percolarea este o tranziție de fază: în jurul lui pc apare un grup de copaci conectat care traversează sistemul; pe o pădure infinită saltul ar fi o treaptă.

10. c) – Ambele modele conservă masa și impulsul; rețeaua pătrată are însă direcții privilegiate, iar simetria hexagonală este suficientă pentru ecuațiile Navier–Stokes la scară mare.

11. a) – Particulele care lovesc obstacolul își inversează direcția (f_i ia valoarea direcției opuse); efectul macroscopic este condiția de nealunecare la perete.

12. d) – LJ scade ca r^{-6}, deci contribuția particulelor îndepărtate este neglijabilă; gravitația are rază lungă, iar contribuțiile îndepărtate nu pot fi ignorate – de aici Barnes–Hut.

13. b) – Pe orbita Kepler, eroarea energiei Verlet rămâne aceeași de la o orbită la alta, în timp ce RK4 derivă liniar, deși costă de 4 ori mai mult pe pas.

14. c) – Fiecare corp parcurge arborele și interacționează cu O(log N) celule; în simulările noastre, interacțiunile pe corp cresc cu o constantă la fiecare dublare a lui N.

15. a) – Fiecare termen Fibonacci depinde de cei anteriori, deci calculul este secvențial; celelalte sunt independente la un pas dat (Cernazean L2).

16. 2 – Accelerare: min(4 + 1, 5) = 5; frânare: min(5, 2) = 2; deci avansează 2 celule.

17. 27 – Fiecare dintre cele 3 celule are 3 stări: $3^3 = 27$ de configurații (iar numărul de reguli ar fi 3^{27}).

18. 0,0025 – $\Delta t \leq \Delta x^2/(4D) = 0{,}01/4 = 0{,}0025$.

19. 0,0556 – $\nu = (0{,}6667 - 0{,}5)/3 = 0{,}1667/3 \approx 0{,}0556$.

20. 10,26 – $S = 1/((1 - 0{,}95) + 0{,}95/20) = 1/(0{,}05 + 0{,}0475) = 1/0{,}0975 \approx 10{,}26$; plafonul pentru N foarte mare ar fi 20.

Bibliografie

- Prof. Cernazean – Modelling and Simulation, Lecture 2 (high-performance computing), Lecture 4 (cellular automata), Lecture 5 (lattice Boltzmann), Lecture 6 (particles) (materiale de curs).
- B. Chopard, M. Droz – Cellular Automata Modeling of Physical Systems, Cambridge University Press, 1998.
- S. Wolfram – Statistical mechanics of cellular automata, Reviews of Modern Physics, 55, 1983; A New Kind of Science, Wolfram Media, 2002.
- M. Gardner – The fantastic combinations of John Conway's new solitaire game „life", Scientific American, 223, 1970.
- K. Nagel, M. Schreckenberg – A cellular automaton model for freeway traffic, Journal de Physique I, 2, 1992.
- Y. Sugiyama et al. – Traffic jams without bottlenecks, New Journal of Physics, 10, 2008.
- D. Stauffer, A. Aharony – Introduction to Percolation Theory, ed. 2, Taylor & Francis, 1994.
- T. Krüger et al. – The Lattice Boltzmann Method: Principles and Practice, Springer, 2017.
- M. P. Allen, D. J. Tildesley – Computer Simulation of Liquids, ed. 2, Oxford University Press, 2017.
- L. Verlet – Computer „experiments" on classical fluids, Physical Review, 159, 1967.
- J. Barnes, P. Hut – A hierarchical O(N log N) force-calculation algorithm, Nature, 324, 1986.
- G. M. Amdahl – Validity of the single processor approach to achieving large scale computing capabilities, AFIPS Conference Proceedings, 30, 1967.
- TOP500 – lista din iunie 2026, top500.org.
- Material realizat cu ajutorul Claude Fable 5.1 și Claude Opus 5.5.